

西南交通大学



应用软件综合课程设计 I

实验报告——系统需求分析

项目名称:	船舶装备故障诊断智能问答系统
专业年级:	软件工程专业 2023 级
姓名 (学号):	任安宁 (2023112565)
指导老师 (职称):	韩敏 (教授)
提交日期:	2026 年 5 月 8 日

目 录

项目名称:	1
船舶装备故障诊断智能问答系统——知识图谱构建模块	1
专业年级:	1
软件工程专业 2023 级	1
姓名 (学号):	1
任安宁 (2023112565)	1
指导老师 (职称):	1
韩敏 (教授)	1
提交日期:	1
2026 年 5 月 8 日	1
1、前言.....	4
1.1 编写目的	4
1.2 读者对象	4
1.3 术语与缩写解释	5
1.4 流程图标定义	7
1.5 参考资料	7
2、任务概述	10
2.1 知识图谱构建	12
2.1.1 流程图	错误!未定义书签。
2.1.2 流程说明	错误!未定义书签。
2.1.3 作业分解	错误!未定义书签。
3、用例分析	14
3.1 用例图	错误!未定义书签。
3.2 用例说明	错误!未定义书签。
3.2.1 知识图谱构建相关用例	17

1、前言

1.1 编写目的

本文的编写目的是为船舶装备故障诊断智能问答系统中的知识图谱构建模块提供系统需求分析的全面框架，以确保各利益相关者对知识图谱构建的功能、流程和技术需求达成共识。知识图谱作为系统的核心知识基础设施，承担着将非结构化故障文档转化为结构化知识的关键任务，是实现精准故障诊断与智能问答的基础。

随着船舶装备日益复杂化，故障诊断涉及发动机、液压系统、电气设备等多学科知识，传统运维模式存在知识孤岛严重、专家资源稀缺、知识传递效率低等痛点。知识图谱技术能够将分散的故障案例、维修手册、操作规范等非结构化知识进行结构化表示，建立实体之间的语义关联，为智能问答系统提供可解释、可追溯的知识支撑。

本需求分析文档将详细描述知识图谱构建模块的各项功能，包括文档预处理、语义分块、实体抽取、关系抽取、实体合并、图谱导入、可视化展示等，并明确各子模块之间的关系和交互方式。模块将设计为可扩展的，以便未来根据新增装备类型和故障案例进行知识图谱的快速更新与扩展。

在编写过程中，我们还将结合知识工程与软件工程的最佳实践，确保知识图谱构建流程的自动化、标准化与高质量，以最大程度上降低人工标注成本，提升知识抽取的准确率与召回率。通过不断的质量评估与反馈优化，知识图谱将在实际运行中不断迭代，提升系统的故障诊断能力与服务可靠性。

1.2 读者对象

本文的主要读者包括但不限于以下人员：

- **项目开发团队：**负责系统的设计、开发和实施，需充分理解系统需求和预期功能，以便高效地进行技术实现。开发团队需具备扎实的软件开发技术背景，并能够将业务需求转化为具体的技术实现方案。

- **项目管理人员：**负责项目的进度控制和资源管理，需关注系统需求的变更对项目进度和预算的影响。项目管理人员需要具备良好的沟通能力，以协调各方资源并解决项目实施中的问题。

- **系统测试人员：**负责系统的测试和验证，需基于文档中的需求进行测试用例的设计和执行，确保系统按预期功能正常运行。测试团队需要深入理解系统的各项功能和业务流程，以便在测试中发现潜在的问题。

- **业务人员和用户代表：**了解系统的实际使用需求，以便为开发团队提供准确的业务反馈和用户体验建议。他们的参与能够帮助开发团队更好地理解用户需求，确保系统的设计能够满足用户期望。

- **维护和支持团队：**负责系统上线后的维护和用户支持，需熟悉系统功能及常见问题的解决方案。维护团队需要能够快速响应用户的问题并提供有效的解决方案，以保证系统的持续运营。

这些读者对象的需求和期望，将在后续的系统设计中起到关键作用。因此，在编写本需求分析文档时，我们特别注意到不同角色的需求，以便提供尽可能全面和详细的信息。

在整个项目过程中，各类读者的反馈至关重要。项目团队应定期与各方进行沟通，以了解需求变化和使用情况，从而及时调整系统设计和功能开发。此外，项目管理者应确保各方对项目进展有清晰的了解，避免因信息不对称而导致的误解和延误。

1.3 术语与缩写解释

(1) 知识图谱 (Knowledge Graph, KG)：用于表示实体、概念及其之间语义关系的有向图结构，是本项目实现故障知识结构化管理与智能推理的核心数据基础设施。

(2) 实体 (Entity)：知识图谱中的基本节点，代表现实世界中的具体对象或抽象概念。在本项目中，实体包括系统 (System)、设备 (Equipment)、组件 (Component)、故障现象 (FaultPhenomenon)、故障原因 (FaultCause)、诊断方法 (DiagnosticMethod)、维修方法

(RepairMethod)、安全注意事项 (SafetyNote)、参数 (Parameter) 等类型。

(3) 关系 (Relation)：连接两个实体的有向边，表示实体之间的语义关联。本项目预定义了 10 类关系：BELONGS_TO (属于)、HAS_COMPONENT (包含组件)、CAUSED_BY (导致)、DIAGNOSED_BY (诊断方法)、REPAIRED_BY (维修方法)、REQUIRES (依赖)、PREVENTS (预防)、CONTROLS (控制)、MONITORS (监控)、CONNECTS_TO (连接)。

(4) 三元组 (Triple)：知识图谱的基本组成单元，由 (头实体, 关系, 尾实体) 构成，用于描述一个完整的事实或知识断言。

(5) RAG (Retrieval-Augmented Generation, 检索增强生成)：一种结合知识库检索与大语言模型生成的技术框架，通过检索相关文档片段增强生成答案的准确性与可追溯性。

(6) Neo4j：一种高性能的图数据库管理系统，采用属性图模型存储数据，支持高效的图遍历查询与复杂关系分析，本项目使用 Neo4j 作为知识图谱的持久化存储引擎。

(7) BGE-M3：由北京智源人工智能研究院开发的多语言文本嵌入模型，支持密集向量检索、稀疏向量检索与多向量检索，本项目使用 BGE-M3 进行实体语义相似度计算与合并。

(8) 语义分块 (Semantic Chunking)：基于文档的标题层级与语义边界进行智能文本分割的技术，相较于固定长度分块能够更好地保留语义完整性，提升后续知识抽取的质量。

(9) 实体合并 (Entity Merging)：解决知识抽取过程中同一实体不同名称、不同写法问题的技术流程，包括精确名称去重与基于嵌入模型的语义相似度合并两个阶段。

(10) LangGraph：基于 LangChain 构建的图结构 workflow 编排框架，支持定义复杂的多步骤数据处理流水线，本项目使用 LangGraph 实现知识图谱构建的端到端自动化。

（11）Chroma：一个开源的向量数据库，用于存储文档块的向量表示并支持高效的语义相似度检索，是本项目 RAG 检索模块的核心组件之一。

（12）SSE（Server-Sent Events）：一种服务器向客户端推送实时数据的技术标准，本项目使用 SSE 实现问答答案的逐 token 流式生成，提升用户交互体验。

（13）离线模式（Offline Mode）：系统在无公网环境下的运行模式，所有模型推理、数据检索与知识图谱查询均在本地完成，无需调用任何云端服务。

（14）增量更新（Incremental Update）：支持在不重建全量索引的前提下，将新增的文档或知识自动融合到现有知识库与知识图谱中的技术机制。

（15）BM25：一种经典的关键词检索算法，基于词频与逆文档频率计算文档与查询的相关性得分，本项目将其与向量检索融合以实现混合检索。

本部分旨在确保所有相关术语和缩写被清晰定义，以避免在后续文档中产生歧义。此外，术语的统一使用将有助于团队成员之间的沟通和理解。项目中所使用的术语和定义应经过团队的共同讨论和确认，确保一致性。

1.4 流程图标定义

流程图的标准化设计不仅可以提升团队协作效率，还能够使外部利益相关者更好地理解系统的运行机制，增强对项目的信心。本需求分析依据表 1 所示的图标定义进行流程图绘制，以便使各方在理解业务流程时能够迅速识别各个流程节点的功能和性质，这将有助于团队成员在不同阶段的工作协调，确保整个开发过程的高效性与透明度。

绘制流程图时，应重视流程的简洁性与可读性，避免不必要的复杂设计。

表 1 流程图标定义

图标	定义	具体描述
----	----	------

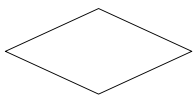
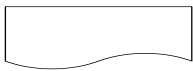
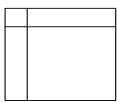
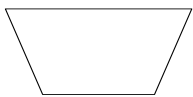
	开始或结束	流程的起始点或终止点
	操作	系统中执行的具体操作步骤
	决定	需要基于条件进行选择的节点
	其它流程	表示流程中的外部数据流
	连接	表示凭证或信息的流转
	打印单据	生成和打印相关的文档
	自动完成	系统自动执行的操作
	手工操作	需要人工干预的操作
	外部数据	需要从系统外接收的数据

图 1-1 流程图标定义

1.5 参考资料

本需求分析报告的参考资料包括以下几类：

（1）标准规范

- 中华人民共和国国家标准《计算机软件开发文件编制指南》
GB8567—88
- 中华人民共和国国家标准《计算机软件需求说明编制指南》
GB8585—88

（2）行业研究资料

涵盖当前船舶运维与故障诊断领域的最新研究成果、技术发展趋势与行业标准，为知识图谱构建提供领域背景信息。这些资料将为系统设计提供依据，以确保知识图谱能够覆盖核心故障场景并适配行业需求。

- 《船舶运维安全白皮书》：由中国船级社于 2024 年发布，详细分析了船舶故障引发的海上事故占比、故障类型分布及预防策略。
- 《船舶装备故障诊断技术研究进展》：发表于《中国航海》期刊，系统综述了船舶故障诊断领域的技术演进与前沿方法。

（3）知识图谱与 RAG 技术资料

对现有知识图谱构建方法、RAG 优化策略及工业领域应用案例进行分析，帮助识别本项目的技术差异化优势，明确技术路线的可行性与先进性。

- 《基于 RAG 技术的工业设备智能问答系统设计》：发表于《计算机工程与应用》期刊，详细介绍了 RAG 在工业设备领域的落地实践与优化策略。
- 《船舶故障知识图谱构建方法研究》：发表于《大连海事大学学报》，系统阐述了船舶领域知识图谱的构建流程与关键技术。

（4）模型与工具官方文档

基于实际技术选型，整理的开源模型、数据库与框架的官方文档，确保系统实现的技术可靠性与版本兼容性。

- Neo4j 官方文档：<https://neo4j.com/docs/>

- LangChain 官方文档：

https://python.langchain.com/docs/get_started/introduction

Meta AI. Llama 3 Open Source Model: <https://ai.meta.com/llama/>, 2024.

（5）其他

- 《知识图谱：方法、实践与应用》- 该书涵盖了知识图谱的构建方法、存储方案与应用场景，可为船舶故障知识图谱的构建提供技术框架参考。

- 《检索增强生成（RAG）技术详解》- 了解 RAG 在垂直领域应用中的关键技术与优化策略。

- 在参考资料的收集过程中，我们将优先选择来自于行业内的权威资料及技术文档，以确保需求分析的准确性和有效性。这些参考资料不仅有助于团队在初期设计时形成共识，还能在后续的开发过程中提供可靠的依据。

- 此外，参考资料的使用还应当注重数据的时效性与准确性，确保文档所依据的信息能够有效指导系统设计和实施。我们将通过建立文献资料库，定期更新相关研究成果和技术动态，确保项目团队始终处于信息的前沿。

2、任务概述

2.1 知识图谱构建

2.1.1 流程图

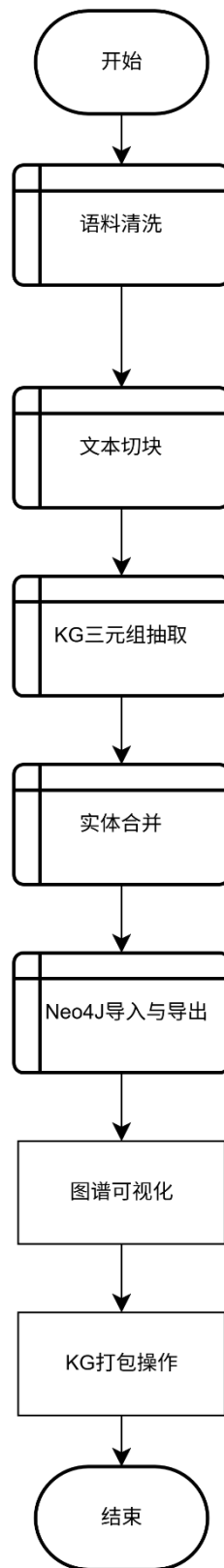


图 2-1 知识图谱构建流程图

2.1.2 流程说明

1. 知识图谱构建流程基于 LangGraph 框架实现端到端自动化，共包含 7 个标准化步骤，支持断点续传与增量更新，实现了从原始非结构化文档到高质量可查询知识图谱的完整转换。

■ 步骤 1：语料清洗。自动识别并去除文档中的目录、版权信息、联系方式、品牌宣传等非技术内容，将 Markdown 标题统一转换为[Hn]格式，提取并本地存储 Base64 编码的图片。通过正则表达式将 HTML 表格转换为文本格式，生成标准化的纯文本语料。

■ 步骤 2：语义分块。解决传统固定长度分块破坏语义完整性的问题，实现基于语义边界的智能分块。优先在 H1/H2 标题处进行分割，将同一系统、同一语义类别的短块进行合并，基于启发式评分过滤低信息密度的空泛概述块，对超长语义块使用递归字符分割器拆分。关键参数：基础块大小 6000 字符，块重叠 360 字符，最小块大小 1200 字符。

■ 步骤 3：KG 三元组抽取。使用大语言模型从文本块中抽取符合船舶领域规范的实体与关系三元组。采用上下文感知抽取策略，利用同文档已抽取实体作为上下文提升一致性；自动将多个小 chunk 打包为批量请求以降低 API 调用次数；使用 LLM 对 chunk 进行质量评分，仅抽取高价值 chunk；自动过滤不符合预定义标签与关系类型的无效结果。

■ 步骤 4：实体合并。解决同一实体不同名称、不同写法的问题，提升知识图谱质量。首先进行精确名称去重，合并名称与标签完全相同的实体；其次统一同一实体的不同标签；然后使用 BGE-M3 嵌入模型计算实体名称语义相似度，合并语义相似实体；最后自动删除节点数小于 5 的孤立连通分量，过滤继电器、保险丝等过于细碎的零件节点。

2. 步骤 5：Neo4j 导入与导出。将清洗后的三元组批量导入 Neo4j 图数据库，建立高效的图索引。采用 CSV 批量导入方式，使用 neo4j-admin import 或 Cypher LOAD CSV 实现百万级三元组秒级导入；采用分批提交策略避免单次事务过大；在实体 name、label 属性上创建唯一索引与全文索引，将查询速度提升 10-100 倍；导入后自动执行统计查询验证数据一致性。

■ 步骤 6：图谱可视化。生成交互式 HTML 可视化图谱页面，支持节点

搜索、关系展开与路径高亮。使用 ECharts 力导向布局实现百万级节点的高性能渲染；根据实体标签设置不同颜色与图标；支持节点点击展开/收起邻居、双击定位实体、输入关键词全局搜索；支持输入两个实体自动高亮最短路径与因果链。输出格式为单文件 HTML（包含所有 CSS/JS）。

■ 步骤 7：KG 打包发布。支持将多个独立构建的子图（如不同系统、不同型号的船舶知识图谱）打包为统一的知识图谱。自动处理同名实体的命名冲突（添加子图名称前缀）；重新分配全局唯一的实体 ID 与关系 ID；自动生成 JSON 格式打包清单，记录子图名称、实体数量、关系数量、构建时间等元信息。

2.1.3 作业分解

作业代码	作业描述	操作员	注释
KG-1	语料清洗 ：自动识别并剔除原始文档中的目录、版权声明、联系方式、多语言冗余及品牌宣传等噪声内容；将 Markdown/数字标题统一标准化为 [Hn] 格式；提取并本地化存储表格与图片附件。	系统自动化执行	1. 清洗内容包含：自动识别并剔除目录、版权声明、联系方式、多语言冗余及品牌宣传等非技术噪声；基于正则将 Markdown/数字标题统一标准化为 [Hn] 格式；自动提取并本地化存储表格与 Base64 图片附件。
KG-STEP-02	文本切块（语义分块） ：基于标题层级与语义边界进行智能分块，合并同语义短块，自动拆分超长块；过滤低信息密度的空泛概述，确保每个块保留完整的故障诊断逻辑。	系统自动化执行	分块规则包含：基于标题层级感知与语义系群合并，优先在 H1/H2 标题处分割；自动合并同语义短块，递归拆分超长块；基于启发式评分过滤空泛概述等低价值块。 2. 参数配置：基础块大小 6000 字符，重叠区 360 字符，最小保留块 1200 字符。
KG-STEP-03	KG 三元组抽取 ：调用大语言模型从分块文本中抽取符合船舶领域规范的实体与关系三元组；自动执行质量前置评分，仅对高价值块进行抽取，严格过滤无效结果。	系统自动化执行	1. 抽取规范包含：严格限定 9 类实体标签（System/Equipment/Component 等）与 10 类关系类型（CAUSED_BY/REPAIRED_BY 等）；禁止抽取继电器、保险丝等细碎零件节点，仅保留核心组件级实体。 2. 质量控制：内置 LLM 对 chunk 进行质量前置评分，仅对高价值块执行抽取；支持同文档上下文感知与分块批量打包处理。
KG-STEP-04	实体合并与去重 ：对抽取结果执行精确名称去重与标签统一；使用 BGE-M3 嵌入模型计算实体名称语义相似度，合并同义异名实体；自动裁剪孤立节点与细粒度零件节点。	系统自动化执行	1. 合并逻辑包含：先执行名称与标签完全一致的精确去重；再调用 BGE-M3 嵌入模型计算实体名称语义相似度，自动合并同义异名/同名异义实体。 2. 过滤策略：自动删除节点数小于 5 的孤立连通分量；强制裁剪细粒度零件节点，有效避免图谱“毛线球”现象。
KG-STEP-05	Neo4j 导入与导出 ：将清洗合并后的三元组转换为 Neo4j 标准 CSV 格式，采用事务分批提交策略批量导入图数据库；在 name 与 label	系统自动化执行	1. 导入机制包含：将三元组转换为 Neo4j 标准 CSV 格式；采用事务分批提交策略（1000 条/批）避免内存溢出；自动处理实体 ID 冲突与重复关系。

	属性创建唯一/全文索引；导出 dump 文件供离线打包。		2. 索引优化：在 name 与 label 属性创建唯一索引与全文索引，查询速度提升 10~100 倍；导入后自动执行实体/关系数量统计校验。
KG-STEP-06	图谱可视化生成： 生成交互式 HTML 静态页面，基于 ECharts 力导向布局渲染节点与关系；支持节点点击展开、邻接高亮、最短路径追踪与关键词全局搜索。	系统自动生成（知识管理员审核）	1. 渲染功能包含：基于 ECharts 力导向布局高性能渲染；根据实体标签自动分配节点颜色与图标；支持节点点击展开/收起邻接关系、双击定位、关键词全局搜索与最短路径高亮追踪。 2. 交付形态：输出为单文件 HTML （内嵌全部 CSS/JS ），无需服务端部署，可直接浏览器打开用于人工质量审核。
KG-STEP-07	KG 打包发布： 将多个独立构建的子知识图谱（如发动机、电气、液压系统）打包为统一交付包；自动处理跨系统实体命名冲突，分配全局唯一 ID ，生成元数据打包清单。	系统自动化执行（项目交付人员）	1. 打包逻辑包含：自动检测跨子图同名实体，添加子系统前缀进行区分（如 主机_冷却系统 ）；重新分配全局唯一的实体 ID 与关系 ID ，消除 ID 冲突。 2. 交付清单：自动生成 JSON 格式打包清单（记录子图名称、实体/关系数量、构建时间戳等元数据）；支持按需组合子系统图谱与热更新。

3、用例分析

3.1 用例图

知识图谱构建模块的用例模型主图如所示。

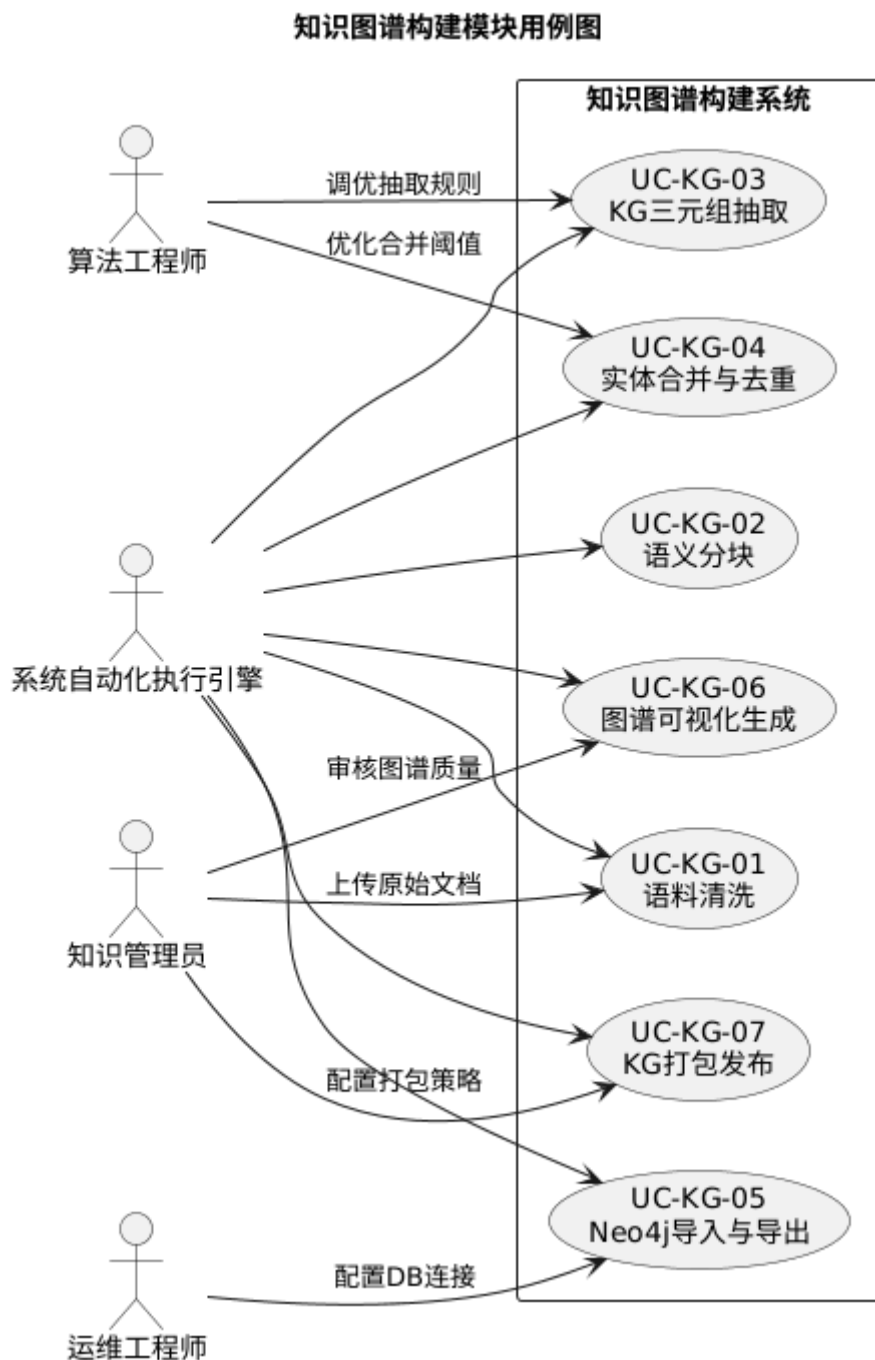


图 3-1 用例图

图 3-1 知识图谱构建用例模型主图

3.2 用例说明

本系统用例说明严格遵循 UML 2.5 规范与软件工程标准模板，从参与者、前置/后置条件、事件流、业务规则及代码映射等维度对核心功能进行结构化定

义。知识图谱构建模块作为系统知识底座的核心引擎，采用 LangGraph 有向无环图（DAG）编排，实现全流程自动化与可观测性。

3.2.1 知识图谱构建相关用例

用例编号	用例名称	主要参与者	优先级	触发条件
UC-KG-01	语料清洗	系统自动化执行（知识管理员监控）	高	原始文档上传至 data/docs/ 目录后自动触发
UC-KG-02	语义分块	系统自动化执行	高	语料清洗完成且状态校验通过后触发
UC-KG-03	KG 三元组抽取	系统自动化执行 + LLM 推理引擎	高	语义分块完成且块数量 ≥ 1 时触发
UC-KG-04	实体合并与去重	系统自动化执行（算法工程师调优）	高	原始三元组抽取完成且文件就绪后触发
UC-KG-05	Neo4j 导入与导出	系统自动化执行（运维工程师配置）	高	实体合并完成且连接参数校验通过后触发
UC-KG-06	图谱可视化生成	系统自动生成（知识管理员审核）	中	Neo4j 导入成功且图谱数据可查询后触发
UC-KG-07	KG 打包发布	系统自动化执行（项目交付人员）	中	可视化生成完成或手动指定子系统组合时触发

【UC-KG-01】语料清洗

项目	内容
前置条件	1. 用户已通过增量上传接口或本地目录 放置原始文档（PDF/MD/TXT） 2. 系统运行环境就绪，依赖库加载成功
后置条件	1. 输出标准化纯文本至 <code>cleaned/</code> 目录 2. 生成清洗日志 <code>pipeline_*.log</code> 与构建状态文件 <code>kg_state.json</code>
基本事件流	1. 系统读取文档，自动识别编码与格式类型 2. 调用 <code>clean_text()</code> 执行噪声剔除：移除目录、版权声明、联系方式、多语言冗余及品牌宣传内容 3. 将 Markdown/数字标题统一标准化为 <code>[Hn]</code> 格式 4. 提取文档内表格与 Base64 图片，本地化存储至 <code>images/</code> 目录 5. 输出清洗后纯文本，记录处理耗时与过滤条目数
备选/异常流	A1. 编码识别失败：自动尝试 UTF-8/GBK/GB2312 依次解码，仍失败则跳过并记录警告 E1. 文件读取异常：记录 <code>ERROR: File read failed</code> ，通知知识管理员手动介入
业务规则	1. 严格保留所有技术术语、故障描述、维修步骤、安全规范等核心内容 2. 禁止删除任何包含“故障”“诊断”“维修”“参数”“报警”等关键词的段落

【UC-KG-02】语义分块

项目	内容
前置条件	UC-KG-01 执行成功，输出标准化纯文本
后置条件	1. 生成结构化分块列表 <code>chunks.json</code> 2. 建立 <code>chunk_id ↔ doc_id ↔ heading_context</code> 映射关系
基本事件流	1. 调用 <code>build_semantic_chunks()</code> 函数 2. 基于 <code>[H1]/[H2]</code> 标题层级进行一级逻辑分割 3. 对同语义短块执行系群合并，对超长块执行递归字符拆分 4. 基于启发式评分过滤低信息密度块（空泛概述、纯目录等）

	5. 输出分块结果，确保每个块保留完整的“故障现象→原因→措施”逻辑链
备选/异常流	A1. 文档无标题层级：退化为基于语义相似度的滑动窗口分块策略 E1. 分块后总数为 0：自动放宽最小区阈值至 600 字符重试
业务规则	1. 基础块大小：6000 字符；重叠区：360 字符；最小保留块：1200 字符 2. 低价值块过滤率需 $\geq 80\%$ ，确保后续抽取聚焦高价值技术内容

【UC-KG-03】KG 三元组抽取

项目	内容
前置条件	UC-KG-02 执行成功，chunks.json 就绪；LLM 服务（Ollama/ModelScope）可调用
后置条件	1. 生成原始三元组集合 kg_raw.json 2. 记录抽取质量评分与无效结果过滤日志
基本事件流	1. 调用 process_extract_chunk_node() 函数 2. 对每个分块执行质量前置评分，仅对高价值块触发抽取请求 3. 调用 LLM，按预定义规范抽取实体与关系： - 实体标签（9 类）： System/Equipment/Component/FaultPhenomenon/FaultCause/DiagnosticMethod/RepairMethod/SafetyNote/Parameter - 关系类型（10 类）： BELONGS_TO/HAS_COMPONENT/CAUSED_BY/DIAGNOSED_BY/REPAIRED_BY/REQUIRES/PREVENTS/CONTROLS/MONITORS/CONNECTS_TO 4. 自动过滤不符合标签规范或包含细碎零件（如继电器、保险丝）的无效结果 5. 输出三元组集合，每条包含 head、relation、tail、source_chunk_id
备选/	A1. LLM 响应超时：自动重试 2 次，仍失败则标记为“待人工复核”并跳过 E1. 高质量块抽取结果为空：记录异常日志，触发告警通知算法工程师

异常流	
业务规则	1. 严格限制实体粒度为“组件级及以上”，禁止抽取接线端子、螺丝等细碎节点 2. 同一文档内抽取需保持实体命名一致性，支持同文档上下文感知

【UC-KG-04】实体合并与去重

项目	内容
前置条件	UC-KG-03 执行成功，kg_raw.json 就绪
后置条件	1. 生成清洗后的高质量三元组 kg_cache_cleaned.json 2. 输出实体合并日志 entity_merge_log.json
基本事件流	1. 调用 merge_kg() 函数 2. 执行精确去重：合并 name+label 完全一致的实体 3. 调用 BGE-M3 嵌入模型计算实体名称语义相似度，合并相似度 ≥ 0.92 的同义实体 4. 自动裁剪节点数 < 5 的孤立连通分量与细粒度零件节点 5. 更新关系中的实体 ID 引用，保证图谱连通性
备选/异常流	A1. 语义合并误判：算法工程师可配置 merge_threshold 参数后重新执行 E1. 合并后关系断裂：自动记录断裂关系，生成修复建议清单供人工审核
业务规则	1. 合并后实体保留信息最丰富的描述字段 2. 禁止合并不同装备系统的同名实体（需通过子系统前缀区分）

【UC-KG-05】Neo4j 导入与导出

项目	内容
前置条件	UC-KG-04 执行成功；Neo4j 服务已启动，连接参数配置正确
后置条件	1. 三元组成功导入 Neo4j 图数据库 2. 创建 name 唯一索引与全文索引 3. 导出 dump 文件供离线部署
基本事件流	1. 调用 import_to_neo4j() 函数 2. 将三元组转换为 Neo4j 标准 CSV 格式 (entities.csv / relations.csv) 3. 采用事务分批提交策略 (1000 条/批) 批量导入，避免内存溢出 4. 自动处理实体 ID 冲突与重复关系，记录冲突日志 5. 在 name 与 label 属性创建唯一索引 + 全文索引，查询速度提升 10~100 倍 6. 导入后执行统计校验：MATCH (n) RETURN count(n) 验证实体数量一致性
备选 / 异常流	A1. Neo4j 连接失败：自动重试 3 次，仍失败则导出 CSV 供离线手动导入 E1. 导入过程中断：记录断点位置，支持从断点续传导入
业务规则	1. 导入前自动备份现有图谱数据至 backups/ 目录 2. 索引创建仅在首次导入时执行，增量更新时跳过以节省时间

【UC-KG-06】图谱可视化生成

项目	内容
前置条件	UC-KG-05 执行成功，图谱数据可查询
后置条件	1. 生成交互式 HTML 可视化文件 kg_vis_*.html 2. 支持浏览器直接打开，无需服务端部署

基本事件流	1. 调用 <code>generate_kg_html()</code> 函数 2. 从 Neo4j 查询实体与关系数据，按标签分类设置节点颜色/图标 3. 基于 ECharts 力导向布局渲染节点-关系图，支持：节点拖拽、缩放、悬停详情、关键词搜索、最短路径高亮 4. 输出为单文件 HTML（内嵌全部 CSS/JS），存储于 <code>data/visualizations/</code> 目录
备选/异常流	- A1. 节点数 > 500：自动启用聚类布局，避免渲染卡顿 - E1. ECharts 渲染异常：降级为静态 SVG 图片输出，并记录警告日志
业务规则	1. 节点标签映射：System=蓝色 / Equipment=绿色 / Component=橙色 / Fault=红色 2. 关系线宽随置信度动态调整，支持故障链路自动追踪

【UC-KG-07】KG 打包发布

项目	内容
前置条件	多个子系统知识图谱已独立构建完成（如发动机/电气/液压）
后置条件	1. 生成统一交付包 <code>kg_delivery_vX.X.zip</code> 2. 输出打包清单 <code>package_manifest.json</code>
基本事件流	1. 调用 <code>package_kgs.py</code> 主程序 2. 自动检测跨子图同名实体，添加子系统前缀区分（如 主机_冷却系统） 3. 重新分配全局唯一的实体 ID 与关系 ID，消除 ID 冲突 4. 合并多个子图的三元组，生成最终 <code>kg_merged.json</code> 5. 自动生成打包清单：记录子图名称、实体/关系数量、构建时间戳、校验哈希值
备选/异常流	- A1. 用户指定仅打包部分子系统：按配置筛选子图，跳过未选中模块 - E1. 打包后校验失败：自动回滚至打包前状态，记录详细错误日志
业务规则	1. 交付包必须包含：知识图谱数据、可视化文件、部署说明、版本清单 2. 支持按需组合子系统，实现知识库模块化交付与热更新